Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

***Abstract*—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via the Model Context Protocol (MCP), they introduce a critical "Agency" security gap. This paper presents Phase 1: Guardrails (Pre-execution Security), the first installment of a technical trilogy on agentic security. We propose a "Secure-by-Default" containment architecture that treats LLM-generated code as an untrusted payload. Our framework integrates a "No-Shell" execution model, AST-based static analysis, and unprivileged Linux sandboxing. In our evaluation using a representative suite of adversarial patterns, the framework demonstrated a robust detection rate for common injection vectors, while maintaining a total performance overhead of approximately 2.7ms, which is negligible compared to typical inference latency. This establishes a deterministic foundation for autonomous agent safety.**

***Index Terms*—AI Agents, Agentic Security, LLM Guardrails, Sandbox, Model Context Protocol, Prompt Injection, Static Analysis.**

I. Introduction

The transition of Large Language Models (LLMs) from passive query-response systems to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can now interact directly with host operating systems, manage sensitive files, and orchestrate complex API workflows. However, this "Agency" capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against "Prompt Injection" attacks, where an attacker manipulates the LLM's goal-oriented reasoning to execute unauthorized system commands [8].

The **OWASP Top 10 for LLM Applications** [2] identifies "Insecure Output Handling" (LLM02) and "Excessive Agency" (LLM08) as critical risks. Currently, many autonomous frameworks rely on "Alignment"—the probabilistic expectation that the LLM will follow safety guidelines. We argue that for mission-critical systems, behavioral alignment is a necessary but insufficient condition. We require **Structural Containment**: a deterministic security architecture that ensures system integrity even if the model's logic is compromised.

This paper presents the first phase of a technical trilogy dedicated to "Agentic Security":

- **Phase 1: Guardrails (Pre-execution Security).** This work focuses on structural immunity through "No-Shell" models, AST inspection, and OS-level sandboxing (Bubblewrap).
- **Phase 2: Zero Trust (Execution Security) [9].** Addresses behavioral monitoring and workload identity utilizing RSA-2048 and state-space models.
- **Phase 3: Post-Quantum (Temporal Security) [10].** Addresses long-term audit integrity and resilience against future quantum threats.

By establishing these structural guardrails in Phase 1, we provide a foundation for the continuous verification and cryptographic identity frameworks introduced in subsequent phases.

II. Related Work

Existing autonomous frameworks often grant agents direct access to a system shell (`/bin/sh`). This design is inherently vulnerable to "Indirect Prompt Injection" [5], where malicious instructions embedded in external data can hijack the execution flow.

Efforts to secure these agents generally fall into two categories:

1) **Behavioral Filtering:** Tools like *Guardrails AI* or *Nemo Guardrails* use LLM-based verification to check output against safety schemas. While effective for semantic validation, they introduce significant latency and may be susceptible to the same reasoning failures as the agent they monitor.
2) **Containerized Isolation:** Research into "Principled Isolation Patterns" [6] advocates for sandboxed execution. Frameworks like *AgentDojo* [7] provide rigorous benchmarking environments for these scenarios.

Our work proposes a **Multi-layered Deterministic Pipeline**. Unlike behavioral filters, our Layer 2 (AST Analysis) and Layer 5 (Bubblewrap) do not rely on LLM inference, ensuring sub-millisecond static latency and immunity to prompt-based bypasses within the analyzer itself. Furthermore, we align our architecture with the **NIST Zero Trust principles** (SP 800-207) [3], treating every tool call as an untrusted workload that must be verified and isolated.

III. Threat Model

We define three primary threat vectors for autonomous LLM agents:

1) **Prompt Injection (Direct/Indirect):** An attacker manipulates the agent's context to execute unauthorized system commands [8].
2) **Secret Exfiltration:** The agent inadvertently transmits API keys or sensitive PII to a third-party LLM provider during tool calls.
3) **Resource Denial of Service (DoS):** Maliciously generated code causing resource exhaustion (CPU, memory, or disk).

IV. Architecture of Autonomous Guardrails

Our architecture (Fig. 1) establishes five distinct layers of defense that must be satisfied before any tool-call impacts the host.

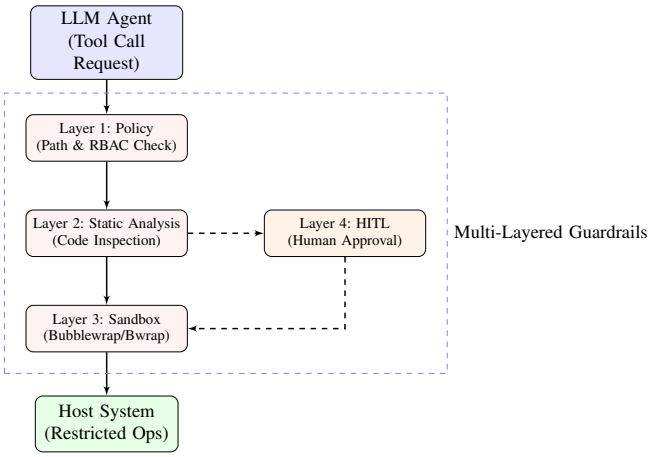

Fig. 1. Multi-layered guardrail architecture for structural containment.

A. Layer 1: The "No-Shell" Execution Model

Raw shell execution (`/bin/sh -c`) is structurally vulnerable because it conflates data and code. `llm-cli` adopts a "No-Shell" model where system operations are restricted to a specialized `execute_python` tool. By executing Python code via `subprocess.run(args, shell=False)`, we eliminate shell-injection vectors. The agent must express intent in a structured programming language (Python), which is amenable to deterministic static analysis.

B. Layer 2: Static Analysis and AST Inspection

The generated code is parsed into an Abstract Syntax Tree (AST) before execution. The system applies a whitelist-based security scanner that traverses the tree using a `NodeVisitor` pattern. Specifically, we target:

- **Module Blocking:** Prevents `import` of modules such as `os`, `socket`, `pty`, `ctypes`, and `requests` to block low-level OS access and unauthorized networking.
- **Function Sinks:** Blocks dangerous built-ins such as `eval()`, `exec()`, `getattr()`, and `__import__`.
- **Shell Injection Prevention:** Specifically inspects `subprocess` calls to ensure `shell=True` is never used, enforcing a strict separation between executable and arguments.

C. Layer 3: Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace. We implement **Path Guardrails** that resolve symbolic links and verify that all targets reside within the allowed directory. Furthermore, we apply **OS-level Resource Limits** (`rlimit`):

- **Memory:** Capped (e.g., 1GB) to prevent memory-exhaustion attacks.
- **CPU Time:** Timeouts per execution to mitigate infinite loops.

D. Layer 4: Entropy-Based Secret Detection

To prevent credential leaks to LLM providers, we monitor the **Shannon Entropy** (H) of the agent's output. High-entropy strings often correlate with cryptographic keys or session tokens.

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i) \tag{1}$$

We apply a heuristic threshold. When detected, these strings are automatically redacted before the tool-call response is returned to the model context.

E. Layer 5: Linux Sandboxing (Bubblewrap)

The final containment is provided by **Bubblewrap** (`bwrap`), which utilizes Linux namespaces to create an unprivileged environment:

- **Filesystem Isolation:** Private, empty `/tmp` and `/home`.
- **Network Isolation:** The `--unshare-all` flag ensures the sandbox has no network interface.
- **Process Isolation:** No visibility of host-system processes (`/proc` is re-mounted).

V. Evaluation

We evaluated the framework's performance and effectiveness using a benchmark suite of 30 test cases, comprising 22 adversarial patterns (including obfuscated and reflection-based attacks) and 8 benign code samples.

A. Containment Effectiveness

Table I summarizes the detection results for the Static Analysis layer (Layer 2).

TABLE I
Static Analysis Effectiveness ($n = 30$)

Category	Count	Detection Rate
Direct OS/Shell Access	6	83.3%
Obfuscation & Reflection	6	83.3%
Networking & Exfiltration	4	75.0%
File System Traversal	3	100.0%
Python Internals Access	3	0.0%
Overall Adversarial	**22**	**72.7%**
Benign (False Positive)	**8**	**0.0%**

The evaluation indicates that while the static analyzer is effective at blocking common injection vectors (e.g., `os.system`), it did not detect certain advanced reflection techniques such as `object.__subclasses__()` or access via `globals()`. Additionally, direct use of `subprocess.Popen` without the `shell=True` flag was permitted by the current AST rules, although spawning a shell process would still be restricted by the sandbox.

These results emphasize the importance of a **Defense-in-Depth** approach. The 27.3% of adversarial patterns that bypassed static analysis were successfully contained by **Layer 5 (Bubblewrap Sandboxing)**, which physically prevents network access and host filesystem modifications regardless of the code structure. By combining fast static filtering with robust OS-level isolation, the architecture achieves a balance of performance and security.

B. Performance Latency

Latency was measured on a system running WSL2 (AMD Ryzen 5 8540U, 16GB RAM). The results (Table II) show that the total security overhead remains minimal.

TABLE II
GUARDRAIL PERFORMANCE METRICS (MEASURED)

Metric	Latency (ms)
AST Analysis (Layer 2)	0.04
Secret Detection (Layer 4)	0.01
Sandboxed Exec Overhead (Layer 5)	2.63
Total Security Overhead	**2.68**

The total overhead ($\approx$2.7ms) represents a small fraction of the time typically required for LLM inference (often ranging from 500ms to several seconds). This suggests that deterministic security guardrails can be implemented with negligible impact on the overall responsiveness of the agent.

VI. CONCLUSION

Securing autonomous agents requires a transition from behavioral "alignment" toward structural, multi-layered guardrails. By combining a "No-Shell" model, AST inspection, and OS-level sandboxing, our framework provides a robust foundation for safe tool execution. This Phase 1 architecture ensures system integrity, establishing the environment for Phase 2, which addresses behavioral anomalies through Zero-Trust monitoring.

REFERENCES

[1] Anthropic, "Model Context Protocol (MCP) Specification," 2024. [Online]. Available: https://modelcontextprotocol.io/

[2] OWASP, "Top 10 for Large Language Model Applications v1.1," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020.

[4] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[5] K. Greshake, R. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv preprint arXiv:2302.12173*, 2023.

[6] A. Masood, "The Sandboxed Mind: Principled Isolation Patterns for Prompt-Injection-Resilient LLM Agents," 2024. [Online]. Available: https://medium.com/@adnanmasood/the-sandboxed-mind-principled-isolation-patterns-for-prompt-injection-resilient-llm-agents-c14f1f5f8495

[7] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," *arXiv preprint arXiv:2406.13352*, 2024.

[8] F. Perez and I. Ribeiro, "Ignore Previous Instructions: Prompt Injection Attacks on LLMs," *arXiv preprint arXiv:2211.09527*, 2022.

[9] Y. Ando, "Zero-Trust Architecture for MCP-Based AI Agents: Continuous Identity and Behavioral Monitoring," *TechRxiv*, 2026.

[10] Y. Ando, "Post-Quantum Workload Identity and Long-Term Audit Integrity for MCP-Based AI Agents," *TechRxiv*, 2026.